

Addtion : Here's a **clear explanation** of your CUDA program that performs **vector addition** on the GPU:

---

## CUDA Vector Addition Explained (Line-by-Line)

---

### Includes

```
cpp
CopyEdit
#include <iostream>
#include <cuda_runtime.h>
```

- `iostream`: For standard C++ input/output.
- `cuda_runtime.h`: Includes CUDA runtime API.

---

### CUDA Kernel

```
cpp
CopyEdit
__global__ void vectorAdd(float *A, float *B, float *C, int N) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (idx < N) {
        C[idx] = A[idx] + B[idx];
    }
}
```

- This function runs on the GPU.
- Each thread computes  $C[i] = A[i] + B[i]$  for its assigned index  $i$ .

---

### Main Function

```
cpp
CopyEdit
int main() {
    int N = 10;
```

- Defines vector size  $N = 10$ .

---

### Memory Allocation (Host + Device)

```
cpp
CopyEdit
size_t size = N * sizeof(float);
```

```
float *A, *B, *C, *d_A, *d_B, *d_C;
A = (float*)malloc(size); // Host memory
B = (float*)malloc(size);
C = (float*)malloc(size);
cudaMalloc(&d_A, size); // Device memory
cudaMalloc(&d_B, size);
cudaMalloc(&d_C, size);
```

---

## Initialize Vectors A & B

```
cpp
CopyEdit
for (int i = 0; i < N; i++) {
    A[i] = i + 1; // A = [1, 2, 3, ..., 10]
    B[i] = (i + 1) * 2; // B = [2, 4, 6, ..., 20]
}
```

---

## Copy to GPU

```
cpp
CopyEdit
cudaMemcpy(d_A, A, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, B, size, cudaMemcpyHostToDevice);
```

---

## Launch Kernel

```
cpp
CopyEdit
int threadsPerBlock = 256;
int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;
vectorAdd<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, N);
```

- Ensures enough blocks to cover all  $N$  elements.
  - Only one block is actually needed for  $N=10$ , but 256 threads/block is safe.
- 

## Copy Result Back

```
cpp
CopyEdit
cudaMemcpy(C, d_C, size, cudaMemcpyDeviceToHost);
```

---

## Print Result

```
cpp
CopyEdit
// Vector A
// Vector B
// C[i] = A[i] + B[i]
```

- Outputs each value of A, B, and their sum in C.

---

## Cleanup

```
cpp
CopyEdit
free(A); free(B); free(C);
cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
```

- Frees memory from both host and device.

---

## ☒ Sample Output

For N = 10:

```
mathematica
CopyEdit
Vector A: 1 2 3 4 5 6 7 8 9 10
Vector B: 2 4 6 8 10 12 14 16 18 20
Calculations (A[i] + B[i]):
C[0] = 1 + 2 = 3
C[1] = 2 + 4 = 6
...
C[9] = 10 + 20 = 30
```

---

## Notes:

- Always check for CUDA errors in real-world programs (`cudaGetLastError()`, `cudaDeviceSynchronize()`).
- For larger N, adjust `threadsPerBlock` and `blocksPerGrid` accordingly.

# MultiPlication :

Here's a breakdown and explanation of your CUDA-based **matrix multiplication** program — well written and efficient for small matrix sizes like  $N = 3$ .

---

## ☒ CUDA Matrix Multiplication (N x N)

---

## Code Walkthrough

---

## Kernel Function: `matrixMul`

```
cpp
CopyEdit
__global__ void matrixMul(float *A, float *B, float *C, int N) {
    int row = threadIdx.y + blockIdx.y * blockDim.y;
    int col = threadIdx.x + blockIdx.x * blockDim.x;
```

- Calculates the global `row` and `col` index for each thread.
- Each thread computes one element `C[row][col]`.

```
cpp
CopyEdit
    if (row < N && col < N) {
        float value = 0;
        for (int k = 0; k < N; k++) {
            value += A[row * N + k] * B[k * N + col];
        }
        C[row * N + col] = value;
    }
}
```

- Inner loop performs the **dot product** of `row` from A and `col` from B.
- 

## Main Logic

```
cpp
CopyEdit
int N = 3;
size_t size = N * N * sizeof(float);
```

- Defines matrix size 3x3.
- 

## Memory Allocation & Initialization

```
cpp
CopyEdit
float *A, *B, *C, *d_A, *d_B, *d_C;
A = (float*)malloc(size); // Host allocation
cudaMalloc(&d_A, size);    // Device allocation
cpp
CopyEdit
for (int i = 0; i < N * N; i++) {
    A[i] = (i % N) + 1;
    B[i] = ((i % N) + 1) * 2;
}
```

- Creates simple values for easier debugging:
  - A: 1 2 3 1 2 3 ...
  - B: 2 4 6 2 4 6 ...

---

## Launching the Kernel

```
cpp
CopyEdit
dim3 threadsPerBlock(16, 16);
dim3 blocksPerGrid((N + 15) / 16, (N + 15) / 16);
```

- Ensures the grid covers all rows/columns.
- For N=3, only one block is launched.

---

## Device to Host Copy

```
cpp
CopyEdit
cudaMemcpy(C, d_C, size, cudaMemcpyDeviceToHost);
```

- After kernel execution, results are copied back to host.

---

## Result Print and Verification

```
cpp
CopyEdit
// Prints A, B, and element-wise computation of C[i][j]
```

- Includes detailed math showing every multiplication step:

```
cpp
CopyEdit
C[0][0] = A[0]*B[0] + A[1]*B[3] + A[2]*B[6] = ...
```

---

## Output Example (for N = 3)

Given A:

```
CopyEdit
1 2 3
1 2 3
1 2 3
```

and B:

```
CopyEdit
2 4 6
2 4 6
2 4 6
```

Expected C (since rows and columns are the same):

```
markdown
CopyEdit
(1*2 + 2*2 + 3*2) = 12
(1*4 + 2*4 + 3*4) = 24
...
```

Each value in  $C[i][j] = (1 + 2 + 3) * B[\text{any column element}] = 6 * \text{col\_multiplier}$ .

So the result:

```
CopyEdit
12 24 36
12 24 36
12 24 36
```

---

## ☒ Final Notes

- ☒ Correct matrix multiplication logic.
- ☒ Proper memory management (host/device).
- ☒ Nicely visualized computation for debugging.
- ⚠ For larger N, shared memory or tiling would be more efficient.